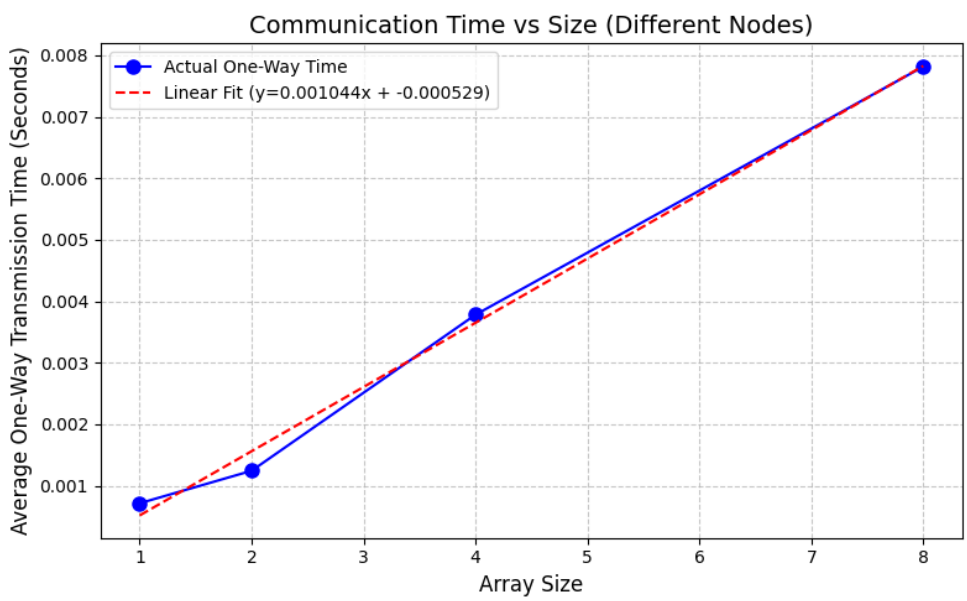
Project 5: MPI Programming (100 points)

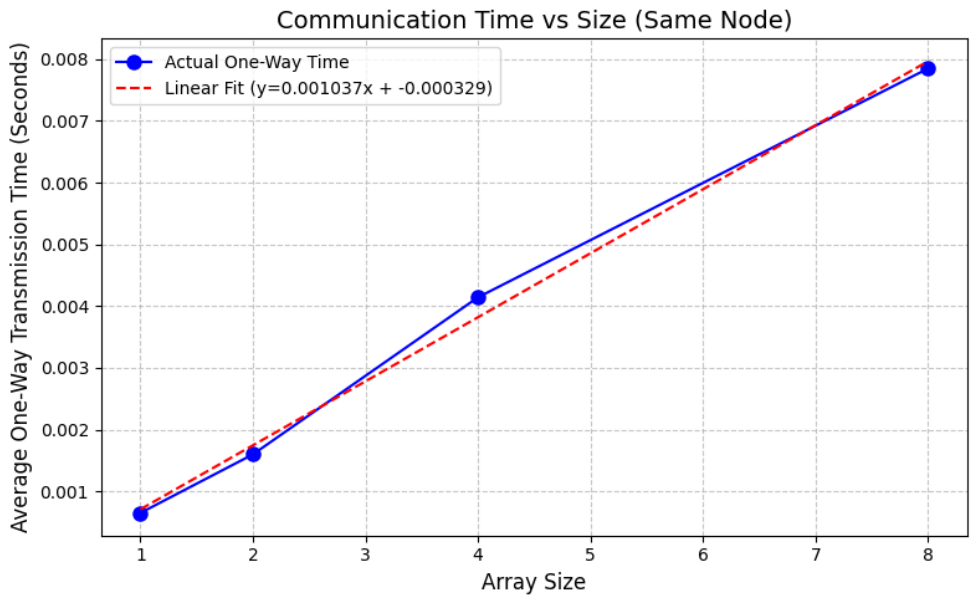
Ephraim Wong
wong0037
113595787

Problem 1. [CS4473 and CS5473] (30 points)

<u>What to submit:</u>

Please submit your MPI code, Makefile and sbatch files and in the report: plot, fitting and estimated values.





	Latency (Seconds)	Bandwidth (MB/s)
Diff Node	-0.000529	3829.46
Same Node	-0.000329	3857.82

Problem 2. [40 points for CS4473 and 20 points CS5473]

<u>What to submit:</u>

Please submit your MPI code, makefile and sbatch file(s) in the ZIP file.

Please report the wall-clock time table, the speedup table and the efficiency table using 2 processes, 4 processes and 8 processes on the "vec1" and "vec2" arrays of length 262144 (2^18), 524288 (2^19) and 1048576 (2^20). The processes can be created on the same node or across multiple nodes.

Array Size	262144	524288	1048576
2 processes	0.003359	0.003276	0.003336
4 processes	0.006911	0.006681	0.006696
8 processes	0.013499	0.013451	0.013478

Please discuss the scalability of the MPI program.

The dot product is a known "embarrassingly parallel pattern", however as seen from the table above, when we increase the number of processes from 2 to 8, the time taken increases for the same array size. In theory, parallelizing this problem should show us a decrease in time taken, however, the adverse observation indicates that the communication overhead from MPI operations is demonstrating negative scalability. This may be due to the simplicity of the problem whereby the array size is too small or the operations are not complex enough for the speedup of parallelization to outperform the overhead of MPI communication. As of this problem statement and test cases, the program does not show positive scalability, however, we need to benchmark computation timings on larger array sizes or more complex operations to determine if this MPI program is truly positively or negatively scalable.

Problem 3. [CS4473 and CS5473] (30 points)

<u>What to submit:</u>

Please submit your MPI code, makefile and sbatch file(s) in the ZIP file.

Please report the wall-clock time table, the speedup table and the efficiency table using 4 processes on the "vec1" arrays of length 262144 (2^18), 524288 (2^19) and 1048576 (2^20). Please benchmark using multiple processes on the same node and then using multiple processes on separate nodes.

You can use "#SBATCH --ntasks-per-node=4" to ask SLURM to run all 4 process in one node and use "#SBATCH --ntasks-per-node=1" to ask SLURM to run only one process per node.

Array Size		262144	524288	1048576
4 Processes on the same node	Wtime()	0.029424	0.029811	0.033085
	Speedup	1.0502	1.03695	0.933677
	Efficiency	0.262549	0.259237	0.23342
4 Processes on 4 different nodes	Wtime()	0.041530	0.04822	0.055143
	Speedup	0.7441	0.640651	0.56034
	Efficiency	0.18602	0.16016	0.14009

Please discuss the scalability of the MPI program under the 2 scenarios (same node vs. different nodes). The discussion should be based on the communication cost that you measured in Problem 1.

The parallel merge sort problem demonstrates a clear difference in scalability. When running 4 processes on the same node, the execution time is low since MPI_Send and MPI_Recv copy data between local RAM allocations. However, when 4 processes are run on different nodes, the execution time increases significantly, which results in a lower speedup and efficiency. This

performance degradation perfectly aligns with the communication overhead observed in Problem 1. However, like in Problem 1, given a large enough problem or more complexity, we can expect to see the performance benefits of parallelization overtake communication costs.